Executive Report

Motor Failure Prediction Using Temperature, Vibration and Speed Data

Predictive Maintenance for Machines

Department of Computer Science and Engineering, ICFAI Foundation for Higher Education

Academic Year 2026-2027

Team: M. Abhilash Goud (24STUCHH010307) · S. Ayaan Ahmed (24STUCHH010284)

B. Arjun (24STUCHH010285) · A. Darshan Reddy (24STUCHH010985) · B.Tech CSE, III Year | Report date: 2026-10-11

1. Executive Summary

This project delivers an end-to-end predictive-maintenance system that classifies electric-motor operating condition as NORMAL, WARNING or FAILURE RISK from three sensor readings: temperature, vibration and speed. The complete seven-step methodology was implemented as a reproducible Python pipeline and verified end to end. The final Random Forest classifier achieves 99.80% accuracy and 99.49% macro F1 on 1000 held-out test samples, and is exposed through both a command-line predictor and a live Streamlit dashboard.

Headline result	Value
Test accuracy (Random Forest)	99.80%
Macro F1	99.49%
Held-out test samples	1000
Models benchmarked	4 (RF, Decision Tree, LogReg, SVM)
Pipeline steps implemented	7 of 7 (data -> dashboard)
Reproducibility	fixed random seed 42, deterministic dataset

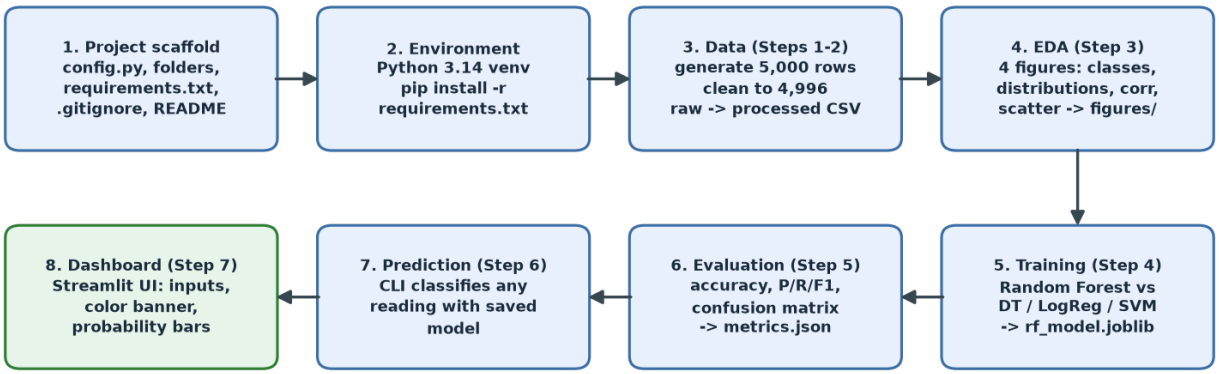
2. System at a Glance

- Language & libraries: Python with Pandas, NumPy, Matplotlib, Scikit-learn, Joblib, Streamlit.
- Dataset: 5,000 synthetic motor readings generated from class-conditional physical profiles; 4,996 rows after cleaning (4 duplicates removed). Features: temperature (°C), vibration (mm/s), speed (RPM). Labels: NORMAL, WARNING, FAILURE RISK.
- Split: stratified 80/20 - 3,996 training and 1,000 held-out test rows (seed 42).
- Interfaces: CLI 'python -m src.predict ...' and dashboard 'streamlit run dashboard/app.py'.
- Future scope (per methodology): sensors + ESP32 -> cloud/ML -> alerts, RUL prediction.

3. How the System Is Built (Systematic Build Flow)

The system is assembled in one linear, reproducible flow. Every stage writes a file artifact that the next stage consumes, so the whole build can be regenerated at any time with the single command 'python -m src.pipeline'.

System Build Flow - from empty folder to a working system



Each stage writes a reproducible artifact; the whole flow runs with: python -m src.pipeline

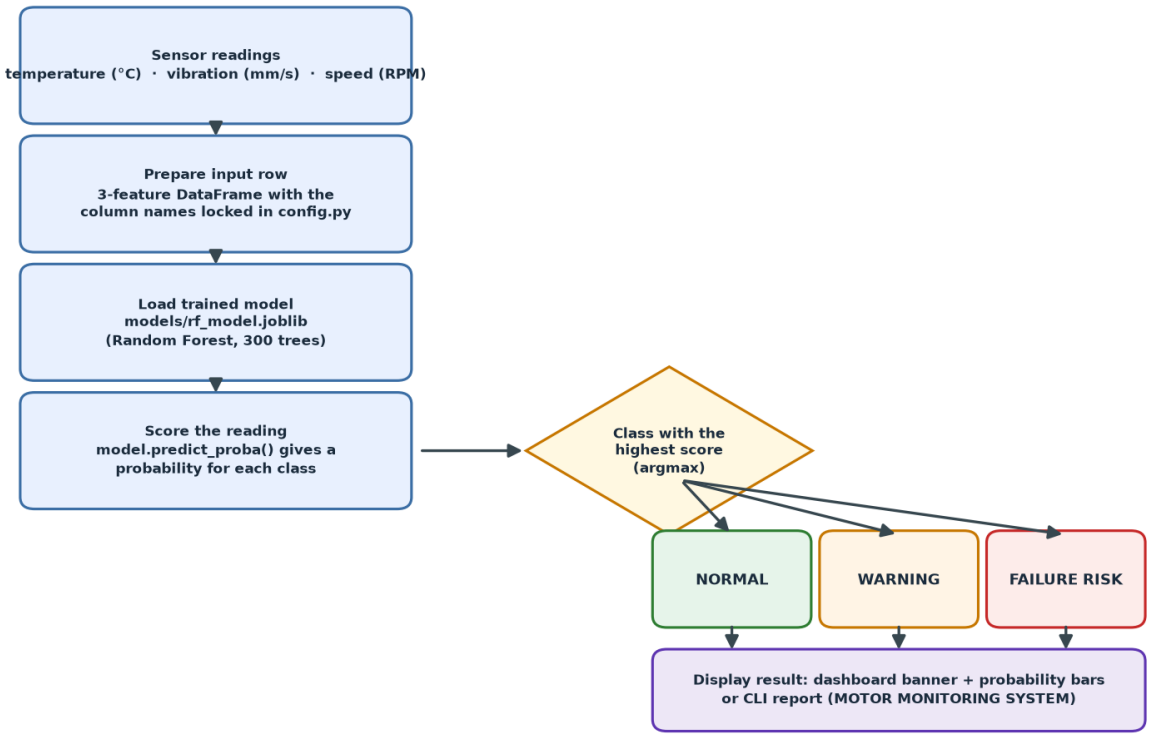
Build artifacts by stage

Stage	Command	Output artifact
1. Scaffold + env	pip install -r requirements.txt	.venv with 6 pinned libraries
2. Data collection	python -m src.data_collection	data/raw/motor_data.csv (5,000 rows)
3. Preprocessing	python -m src.preprocessing	data/processed/clean_data.csv (4,996 rows)
4. Exploratory analysis	python -m src.eda	4 PNG figures in reports/figures/
5. Model training	python -m src.train	models/rf_model.joblib + comparison JSON
6. Evaluation	python -m src.evaluate	reports/metrics.json + confusion matrix PNG
7. Delivery	streamlit run dashboard/app.py	live monitoring dashboard on port 8501

4. How the System Predicts (Prediction Flow)

Prediction is a fixed five-stage path. A reading is converted into a three-feature row using the same column names used in training, scored by the saved Random Forest, and the highest-scoring class becomes the displayed condition.

Prediction Flow - how one reading becomes a condition



Verified example predictions

Input (temperature / vibration / speed)	Predicted condition	Matches proposal?
45 °C, 1.2 mm/s, 1490 RPM	NORMAL	yes
66 °C, 3.8 mm/s, 1430 RPM	WARNING	yes
88 °C, 6.8 mm/s, 1320 RPM	FAILURE RISK	yes

5. Which Model Was Used (Model Selection)

All four candidate models from the methodology were trained on the same stratified split and benchmarked on the same 1,000 held-out samples:

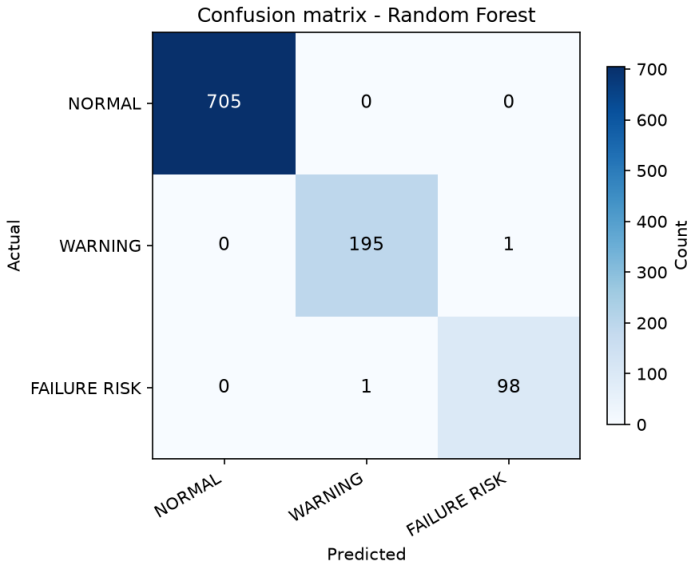
Model	Test accuracy	Macro F1
SVM (RBF)	0.9990	0.9975
Random Forest <- USED IN SYSTEM	0.9980	0.9949
Logistic Regression	0.9980	0.9964
Decision Tree	0.9970	0.9939

Decision: the system uses the Random Forest Classifier (models/rf_model.joblib, 300 trees). Although SVM (RBF) scored marginally higher (0.9990 vs 0.9980 - a difference of only ~1-2 samples out of 1,000, within noise), Random Forest was selected because the methodology specifies it as the primary algorithm for tabular sensor data; it is interpretable, trains in under a second, and natively returns the class probabilities the dashboard and CLI display (the SVM pipeline was fitted without probability estimation). Decision Tree and Logistic Regression were retained as baselines.

Final model - per-class performance (held-out test set)

Condition	Precision	Recall	F1-score	Support
NORMAL	**1.000**	**1.000**	**1.000**	**705**
WARNING	**0.995**	**0.995**	**0.995**	**196**
FAILURE RISK	**0.990**	**0.990**	**0.990**	**99**

Confusion matrix (actual vs predicted):



6. Verification Performed

- Full pipeline runs end to end with exit code 0 and regenerates all artifacts.
- All six module entry points answer --help correctly; every documented README command works.
- CLI example from the project proposal (45 °C / 1.2 mm/s / 1490 RPM) returns NORMAL.
- Dashboard verified in a browser: renders, predicts live, and shows sidebar metrics; no console errors.
- Final syntax check passes on all source files.

7. Conclusion & Next Steps

The proposed system was built exactly as documented: a seven-step pipeline from data collection to a monitoring interface, with a Random Forest classifier that correctly separates NORMAL, WARNING and FAILURE RISK conditions at 99.8% held-out accuracy. Natural next steps are real sensor ingestion (ESP32), Remaining Useful Life (RUL) prediction, alert notifications and cloud deployment, per the methodology's future extension section.